Lycée Duhamel du Monceau. Niveau : Première.		
	Numérique et Sciences Informatiques	
Les tris par sélection et par insertion		

I) <u>LES METHODES DE TRIS PREDEFINIES</u> :

On dispose d'une liste d'objets comparables. Par exemple des nombres, ou des chaînes de caractères.
Il existe bien entendu des méthodes python permettant un tri efficace.

<u>Exemple 1</u> : fonction créant une nouvelle liste.

```
from random import randint

longueur=10
# création d'une liste d'entiers au hasard
L =  [ randint(1,100) for j in range(longueur) ]
print('Contenu de la liste L :\n', L)

# création d'une nouvelle liste
# même contenu que L
# mais triée :
M=sorted(L)
print('Contenu de la liste M=sorted(L) :\n ',M)
# L est inchangée :
print('Contenu de la liste L :\n', L)
```

Ce qui donne :

```
Contenu de la liste L :
 [40, 32, 79, 2, 63, 8, 85, 9, 76, 86]
Contenu de la liste M=sorted(L) :
  [2, 8, 9, 32, 40, 63, 76, 79, 85, 86]
Contenu de la liste L :
 [40, 32, 79, 2, 63, 8, 85, 9, 76, 86]
```

<u>Exemple 2</u> : méthode agissant sur la liste initiale.

```
L =  [ 'choucroute','barbapapa', 'tsointsoin', 'abracadabra' ]
print('Contenu de la liste L :\n', L)

L.sort()

print('Contenu de la liste L :\n', L)
```

L'affichage obtenu :

```
Contenu de la liste L :
 ['choucroute', 'barbapapa', 'tsointsoin', 'abracadabra']
Contenu de la liste L :
 ['abracadabra', 'barbapapa', 'choucroute', 'tsointsoin']
```

L'objectif de cette séance sera de découvrir des algorithmes de tri et non d'utiliser les méthodes déjà programmées.

II) <u>LE TRI PAR SELECTION</u> :

1°) <u>Le tri par sélection du minimum</u> :

Nous exposons ci-dessous le principe du tri par sélection du minimum.

<u>Entrée</u> : une liste d'éléments comparables.

Sortie : la liste triée en ordre croissant.

Traitement :

1. On colore en noir tous les éléments.
2. On repère un élément de valeur minimale parmi les éléments noirs.
3. On passe cet élément en tête des éléments noirs puis on le colore en rouge.
4. On reprend au point 2 tant qu'il reste des noirs.

Illustration.
On applique cette procédure sur la liste L=[3, 2, 1, 5, 4, 2].
On sélectionne le minimum des noirs (ici, il s'agit de L[2]=1). On le passe en tête des noirs et on le colore en rouge.
La liste est maintenant L=[1, 3, 2, 5, 4, 2].
On recommence : on sélectionne un élément de valeur minimale. On a ici deux choix, on choisit l'un des deux au hasard. Par exemple la première occurrence de 2.
La liste est maintenant L=[1, 2, 3, 5, 4, 2].
On continue tant qu'il reste des éléments noirs, ce qui donne les étapes suivantes :
L=[1, 2, 2, 3, 5, 4]
L=[1, 2, 2, 3, 5, 4]
L=[1, 2, 2, 3, 4 , 5]
L=[1, 2, 2, 3, 4, 5]

Application.
Sélection manuelle :
Tri par sélection du minimum.
Soit L=[45, 2, 4, 6, -5, 4, 3], écrire les différents états de la liste L lors du déroulement du tri par sélection.

2°) Le tri par sélection du maximum :
Décrire une procédure de tri (ordre croissant) par sélection du maximum.
Puis l'appliquer à la liste précédente.

3°) Algorithme du tri sélection :
Voici l'algorithme partiel de tri par sélection. Le compléter.

```
Procédure tri_selection(liste L)
    n ← longueur de la lise L
    i ← 0
    Tant que i < n :
        j ← i
        Tant que j < n :
            #
            TO DO
            #
            j ← j + 1
        i ← i + 1
        Fin Tant que
    Fin Tant que
```

4°) Implémentation de l'algorithme :
Implémenter ce code en Python, et vérifier qu'il est bien opérationnel sur la liste L=[10,2,21,12,50,33,22,4,99,9].

5°) Complexité :
Déterminer la complexité de cet algorithme.

III) LE TRI PAR INSERTION :

1°) Présentation :
Le tri par insertion est considéré comme le tri le plus efficace sur des entrées de petite taille. Il est aussi très rapide lorsque les données sont déjà presque triées.
Dans l'algorithme, on parcourt le tableau à trier du début à la fin. Au moment où on considère le *i*-ème élément, les éléments qui le précèdent sont déjà triés. L'objectif d'une étape est d'insérer le *i*-ème élément à sa place parmi ceux qui

précèdent. Il faut pour cela trouver où l'élément doit être inséré en le comparant aux autres, puis décaler les éléments afin de pouvoir effectuer l'insertion. En pratique, ces deux actions sont fréquemment effectuées en une passe, qui consiste à faire « remonter » l'élément au fur et à mesure jusqu'à rencontrer un élément plus petit.

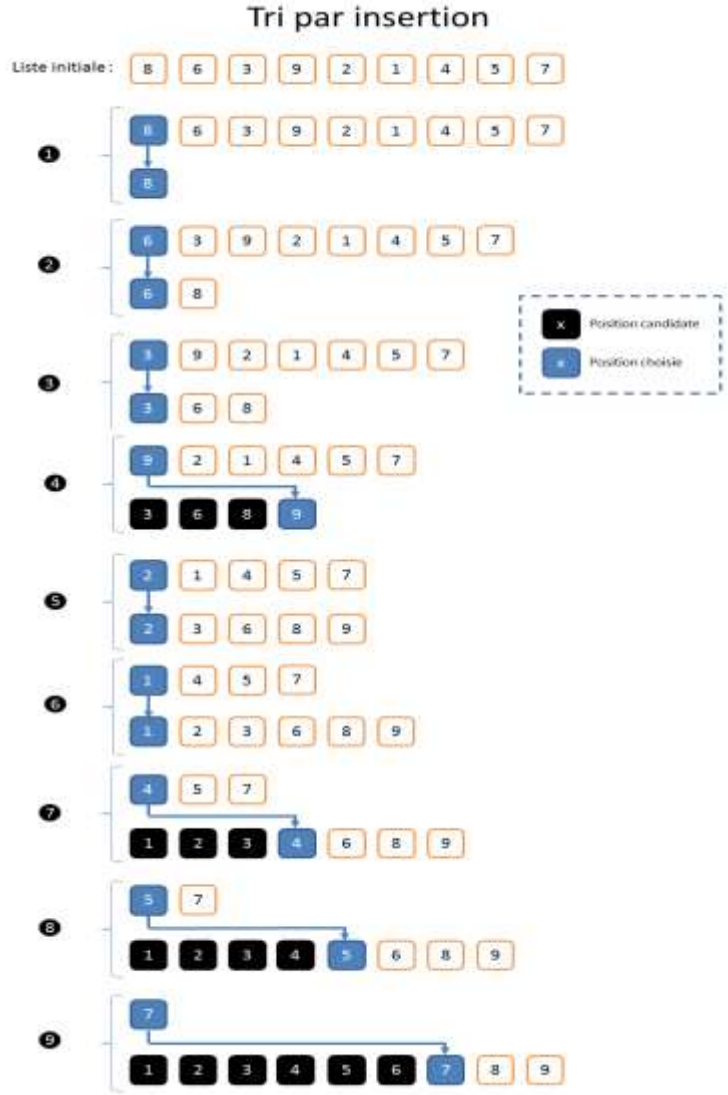
Voici les étapes de l'exécution du tri par insertion sur le tableau [6, 5, 3, 1, 8, 7, 2, 4]. Le tableau est représenté au début et à la fin de chaque itération.

Illustration graphique du tri par insertion.

i = 1 :	6	5	3	1	8	7	2	4	⟶	5	6	3	1	8	7	2	4
i = 2 :	5	6	3	1	8	7	2	4	⟶	3	5	6	1	8	7	2	4
i = 3 :	3	5	6	1	8	7	2	4	⟶	1	3	5	6	8	7	2	4
i = 4 :	1	3	5	6	8	7	2	4	⟶	1	3	5	6	8	7	2	4
i = 5 :	1	3	5	6	8	7	2	4	⟶	1	3	5	6	7	8	2	4
i = 6 :	1	3	5	6	7	8	2	4	⟶	1	2	3	5	6	7	8	4
i = 7 :	1	2	3	5	6	7	8	4	⟶	1	2	3	4	5	6	7	8

2°) <u>Comprendre la procédure</u> :

Vous avez certainement utilisé cet algorithme, c'est celui que les joueurs de cartes utilisent pour organiser leurs mains. Le joueur pioche la carte qui est en haut du tas et la place (l'insère) à la bonne position dans la main. Cet algorithme se nomme le « tri par insertion ». Pour simplifier nous allons trier une liste de 9 valeurs [8, 6, 3, 9, 2, 1, 4, 5, 7].



a) Réaliser le même type de schéma avec [5, 12, 3, 7, 4].

b) Combien de comparaisons faut-il pour trier [8, 6, 3, 9, 2, 1, 4, 5, 7] ?

3°) Algorithme :

L'algorithme partiel de tri par insertion est le suivant. Le compléter.

```
Procédure tri_insertion(liste L)
   n ← longueur de la lise L
   pour j de 1 à n
      key ← L[j]
      i ← j – 1
      tant que i >= 0 et L[i] > key
         #
         TO DO
         #
      fin tant que
      L[i+1] ← key
  fin pour
fin procédure
```

4°) Implémentation de l'algorithme :

a) Implémenter ce code en Python, et vérifier qu'il est bien opérationnel sur la liste L=[10,2,21,12,50,33,22,4,99,9].
b) Décrire le fonctionnement du tri par insertion sur la liste en mettant en évidence les différentes étapes et en matérialisant la position de la variable key.

5°) Complexité :

Déterminer la complexité de cet algorithme.

IV) APPROCHE VISUELLE DE LA COMPLEXITE :

1°) Représentation graphique :

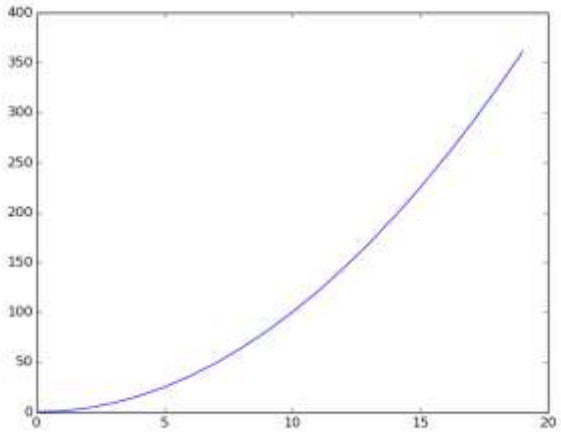
Voici une méthode simple pour une représentation graphique :

```
from pylab import plot, show

X = [ x for x in range(20) ]
Y = [ x**2 for x in X]

plot(X,Y)
show()
```

On obtient la courbe suivante :



Cette courbe est constituée des points de coordonnées (x,y) avec x dans la liste X et y = x**2, élément de même rang, dans la liste Y, que x dans la liste X. Les points sont ensuite reliés. On peut éviter de relier les points et ne laisser réellement que les points définis par les listes X, Y de la façon suivante :

```
from pylab import plot, show
```

```
X = [ x for x in range(20) ]
Y = [ x**2 for x in X]

plot(X,Y,'.') # option points non reliés
show()
```

Nous allons utiliser ici polyfit (from pylab import polyfit) dont le fonctionnement est le suivant :
Soit X une liste de nombres et Y une liste de nombres. polyfit(X,Y, 2) calculera un polynôme du second degré "au plus près de la courbe" définie par les points (x,y). (Nous ne détaillons pas ici plus avant en quel sens "au plus près" doit être compris.)
Avec l'instruction `p=polyfit(X,Y,2)`, le polynôme de degré 2 "au plus près de la courbe" calculé par polyfit sera donné par : `p[0]*x**2+p[1]*x+p[2]`

Vous pouvez par exemple consulter cette page pour en savoir plus sur polyfit.

2°) Durée d'un tri par sélection :

En utilisant la fonction time() (qui permet d'avoir la durée d'exécution d'un programme) du module time et polyfit, nous allons vérifier que le temps d'exécution d'un tri par sélection est fonction du second degré de la taille de la liste donnée en entrée.

Voici le code partiel :

```
from random import randint
from time import time
from pylab import plot, show, polyfit

def creationListe(n) :
        """ crée une liste de n éléments,
        éléments au hasard entre 1 et 2n."""
        return [randint(1,2*n) for j in range(n)]

def triselection(L) :
   """ tri la liste suivant le tri par sélection
   et retourne le temps d'exécution."""
   start=time()
   #
   TO DO
   #
   return time()-start

# liste de tailles de listes :
TailleListe = range(50, 8000, 200)
# liste des temps d'exécution :
TempsTri = [triselection(creationListe(n)) for n in TailleListe]
# approximation par polynôme second degré :
p=polyfit(TailleListe, TempsTri, 2)
Yp= [p[0]*x**2+p[1]*x+p[2] for x in TailleListe]
# courbes  (taille de la liste, Temps d'exécution)
# et (taille de la liste, polynôme second degré approchant) :
plot(TailleListe, TempsTri,'r--',TailleListe, Yp, 'b*')
# on lance une visualisation de la courbe :
show()
```

a) Compléter ce code.
b) Que représentent les courbes obtenues ?
c) Les courbes obtenues sont-elles cohérentes avec la complexité évaluée précédemment ?

3°) Durée d'un tri par sélection :

Pour les plus rapides, faire la même étude pour le tri par insertion.